Universidad del Valle de Guatemala

Departamento de Ciencias de la Computación

Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Redes, sección 10



Proyecto final: Fase 1

Daniel Estrada 20853

André Pivaral 23574

Melisa Mendizabal 23778

Renato Rojas 23813

1. Descripción de la empresa

Mansión del Disfraz es una startup guatemalteca dedicada a la venta en línea de disfraces y decoración temática, con temporada alta en octubre. Todos sus servicios de red se alojan en la nube, sin infraestructura física propia.

Las necesidades de segmentación estimadas por el CEO se traducen en cuatro redes lógicas:

- **Ventas:** 25 personas, encargadas de captar y atender clientes y comercios.
- **TI:** 15 personas, encargadas de desarrollar y mantener la plataforma.
- **Data Center:** 5 servidores (tienda web, base de datos, inventario, pagos y correo).
- **Visitas:** red separada para invitados, sin acceso a recursos internos.

2. Selección del proveedor

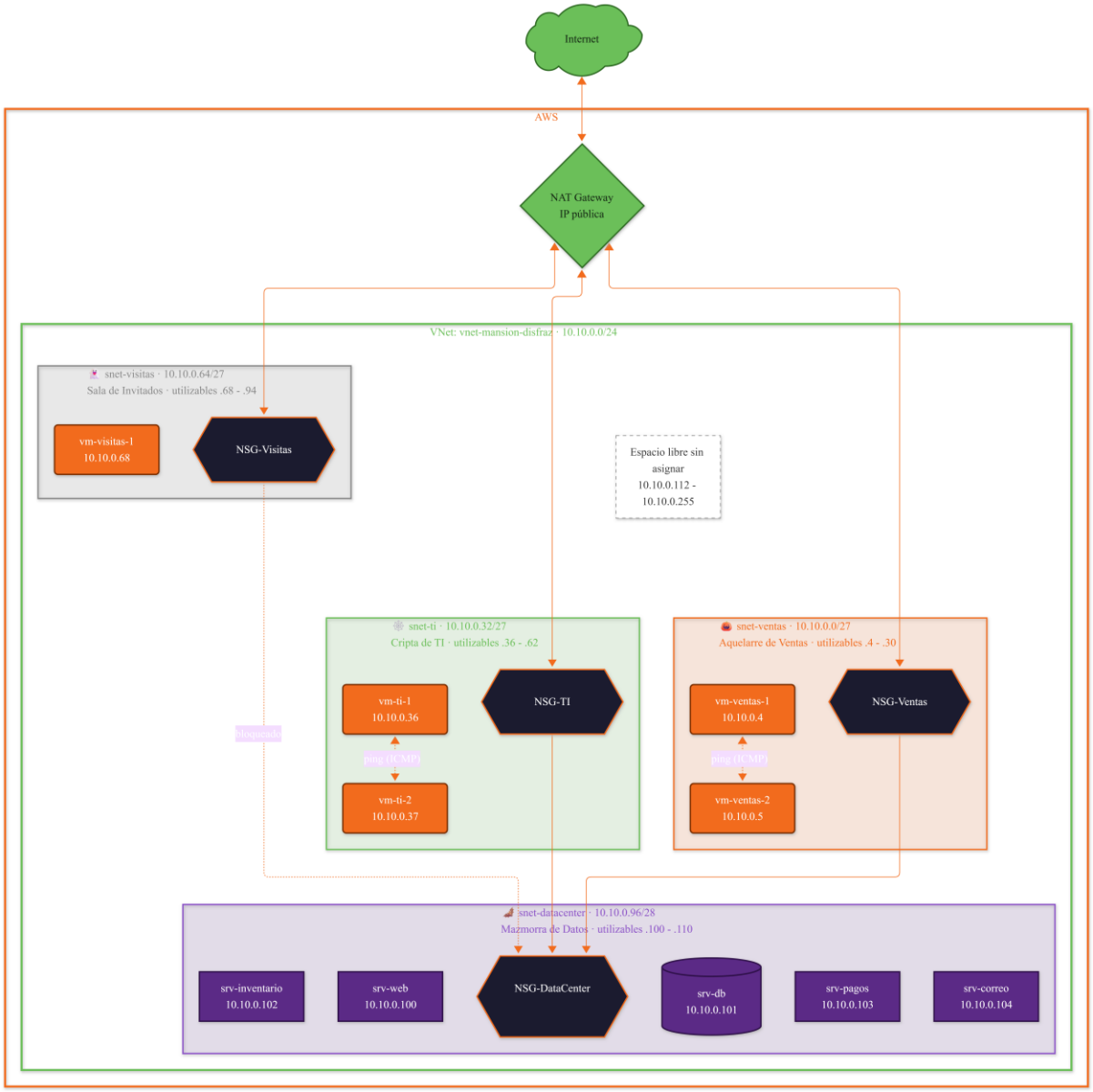
Proveedor elegido: Amazon Web Services.

- AWS organiza la red en una VNet con subredes y grupos de seguridad (NSG), lo que corresponde directamente con la segmentación solicitada.
- Permite aprovechar créditos del free tier, para costear instancias de prueba o negocios iniciando.
- Dispone de portal gráfico y CLI para documentar la implementación con capturas de pantalla.
- Reserva 5 direcciones por subred (red, gateway, dos de DNS y broadcast), dato que se considera en todos los cálculos.
- Tiene una muy sencilla forma de implementación, así como acepta scripts de configuración para la creación de sus instancias/VMs.
- Tiene posiblemente la cuota de mercado más alta entre los servicios, por lo que su comunidad de técnicos y de FAQ es realmente amplia.

3. Diagrama de red

El diagrama muestra la VNet 10.10.0.0/24 con sus cuatro subredes, los NSG asociados, la salida a Internet mediante NAT y las instancias usadas en la prueba de conectividad.

Enlace al diagrama de red: Enlace Diagrama de red



Resumen de subredes y recursos

Subred	Dirección de red	Servidores / instancias (ejemplo)
Ventas	10.10.0.0/27	vm-ventas-1 (10.10.0.4), vm-ventas-2 (10.10.0.5)
TI	10.10.0.32/27	vm-ti-1 (10.10.0.36), vm-ti-2 (10.10.0.37)
Visitas	10.10.0.64/27	vm-visitas-1 (10.10.0.68)
Data Center	10.10.0.96/28	srv-web (.100), srv-db (.101), srv-inventario (.102), srv-pagos (.103), srv-correo (.104)

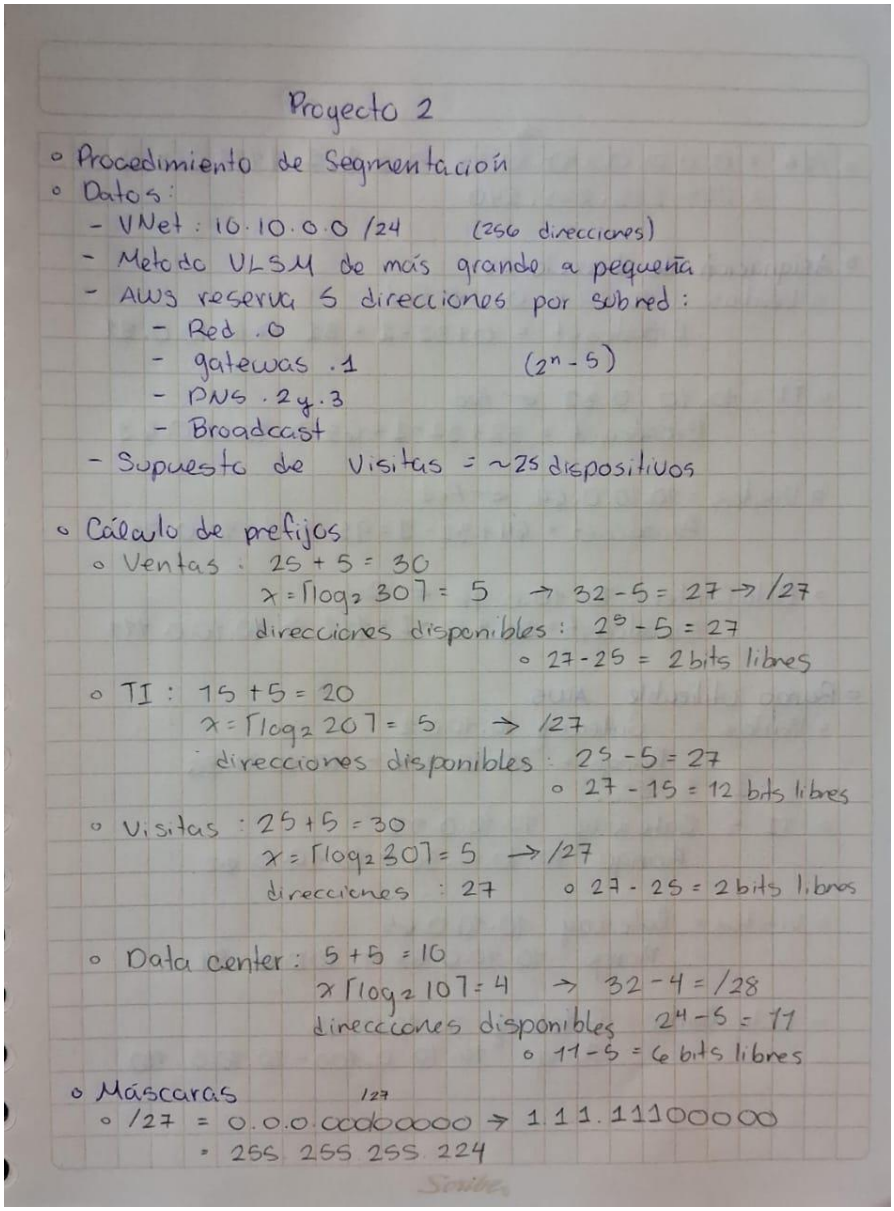
Ventas y TI pueden comunicarse con el Data Center, mientras que Visitas solo tiene salida a Internet.

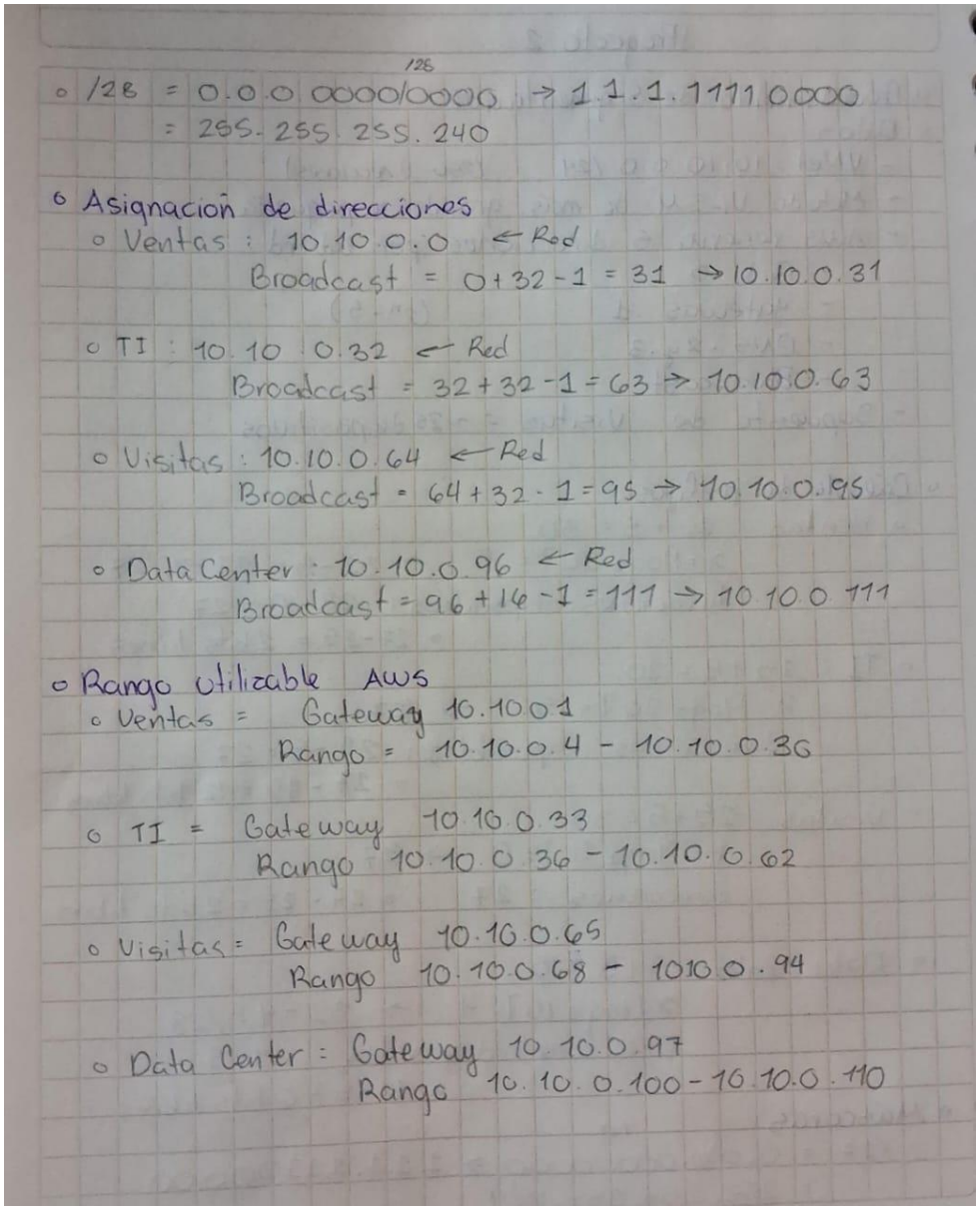
4. Cálculos de segmentación de la red

4.1 Supuestos

- **Espacio de direcciones de la VNet:** 10.10.0.0/24 (256 direcciones).
- **Método:** VLSM, asignando de la subred más grande a la más pequeña.
- **Reservas de AWS:** AWS reserva 5 direcciones por subred: dirección de red (.0), gateway (.1), dos de DNS (.2 y .3) y broadcast (última dirección).
- **Visitas:** el enunciado no indica la cantidad de visitantes; se supone ~25 dispositivos simultáneos.

4.2 Procedimiento





4.2 Resultado del direccionamiento

Subred	Hosts requeridos	Prefijo	Máscara	Dirección de red	Broadcast
Ventas	25	/27	255.255.255.224	10.10.0.0	10.10.0.31
TI	15	/27	255.255.255.224	10.10.0.32	10.10.0.63
Visitas	~25 (supuesto)	/27	255.255.255.224	10.10.0.64	10.10.0.95
Data Center	5	/28	255.255.255.240	10.10.0.96	10.10.0.111

Subred	Rango utilizable en AWS	Gateway (.1)	Total de IPs	IPs reservadas	Utilizables	Libres para crecimiento
Ventas	10.10.0.4 – 10.10.0.30	10.10.0.1	32	5	27	2
TI	10.10.0.36 – 10.10.0.62	10.10.0.33	32	5	27	12
Visitas	10.10.0.68 – 10.10.0.94	10.10.0.65	32	5	27	2
Data Center	10.10.0.100 – 10.10.0.110	10.10.0.97	16	5	11	6

4.3 Espacio sin asignar

Rango libre: 10.10.0.112 – 10.10.0.255 (144 direcciones), equivalente a los bloques 10.10.0.112/28 y 10.10.0.128/25.

Direcciones asignadas: 112 de 256 (43.75 % de la VNet).

4.4 Notas de dimensionamiento

- **TI:** un /28 no es suficiente en AWS, pues ofrece 11 direcciones utilizables y se requieren 15.
- **Data Center:** un /29 solo ofrece 3 direcciones utilizables en AWS y se requieren 5, por lo que se usa /28.
- **Margen:** Ventas y Visitas quedan con solo 2 direcciones libres; si se espera crecimiento, puede ampliarse con el espacio sin asignar.

5. Implementación en la nube

Pasos usados: Crear una cuenta nivel Free Tier en AWS ($100 en 6 meses). Luego, abrir en la Cloud Console los 3 archivos de configuración utilizados ("deploy-aws.sh", "pruebas-aws.sh", "limpiar-aws.sh"), los cuales están en el repositorio de este proyecto (link al final del documento). De esta manera, corriendo los scripts, automáticamente se crean las instancias, VMs, configuraciones de red, etc. necesarias para el prototipado de la red solicitada.

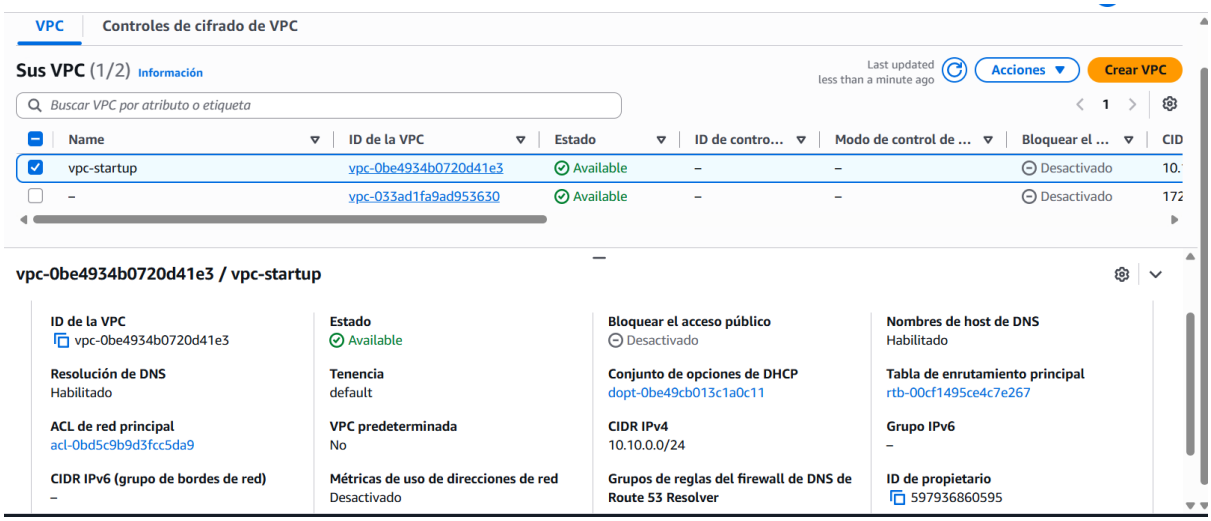
El script final ("limpiar-aws.sh") borra todo lo creado en el primer script; esto se hace de esta manera para ahorrar costos respecto al Free Tier de AWS. Idealmente, se debería dejar corriendo luego del deploy, no obstante, para evitar sobrecargos en la cuenta (la cual sí tiene la tarjeta de crédito asociada), se prefirió crear este script de limpieza. No obstante, primero guarda todos los resultados y todas las configuraciones en JSONs para su posterior análisis, por lo que no se pierde información vital del deploy.

```
~ $ bash deploy-aws.sh
==> Region: us-east-1 | Zona: us-east-1a | Instancia: t3.micro
==> Creando VPC 10.10.0.0/24
==> Creando subredes
==> Creando Internet Gateway y tabla de rutas publica
==> Creando Network ACL nacl-internas (aislamiento de Visitas)
==> Creando Security Group sg-startup
==> Creando llave SSH proyecto-key
==> Lanzando instancias
==> Esperando a que arranquen (1-2 min)

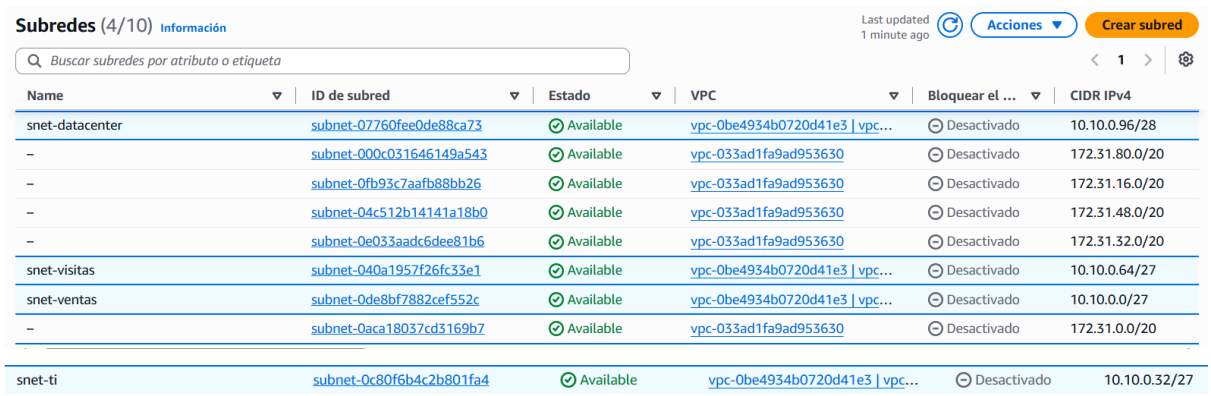
==> Listo. Instancias:
-----------------------------------------------------------------
|                       DescribeInstances                       |
+---------------+-------------+-----------------+-----------+
|  vm-ventas-1  |  10.10.0.15 |  3.239.67.207   |  t3.micro |
|  vm-visitas-1 |  10.10.0.81 |  44.197.216.60  |  t3.micro |
|  vm-ventas-2  |  10.10.0.9  |  34.201.26.98   |  t3.micro |
+---------------+-------------+-----------------+-----------+

Siguiente paso: bash pruebas-aws.sh
~ $
```

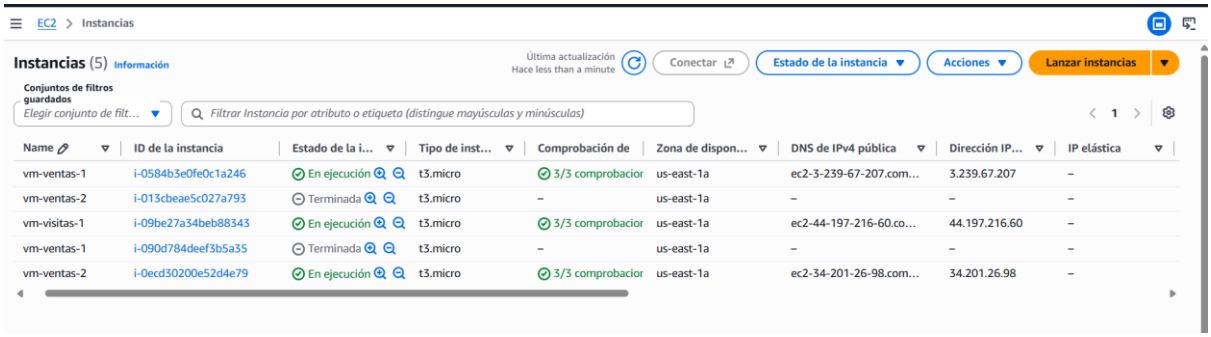
Resultado de correr el script "deploy-aws.sh", dentro de la instancia de AWS. Este archivo, junto con los demás, se encuentra en el repositorio del proyecto (link al final)



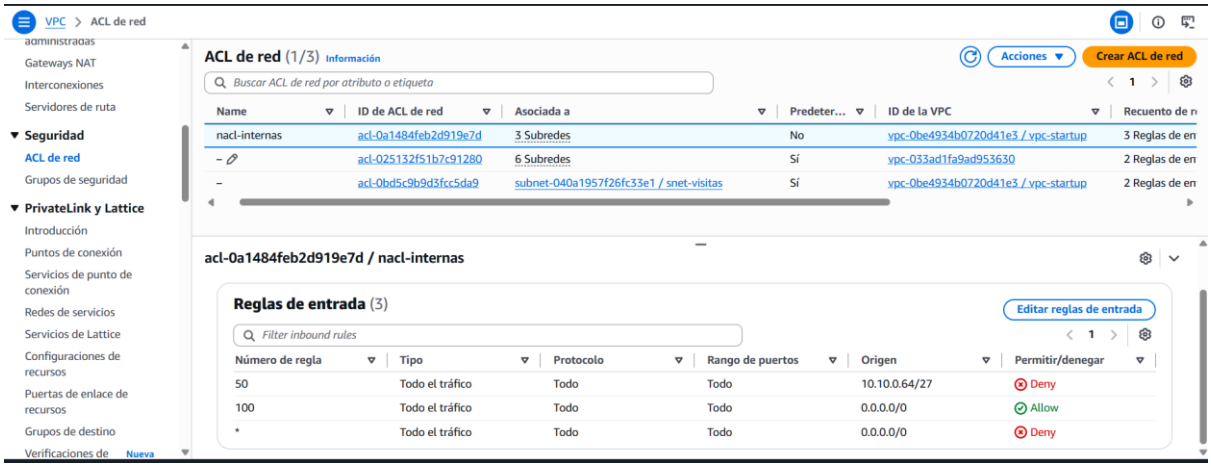
Verificación del estado de la instancia VPC en la consola de AWS



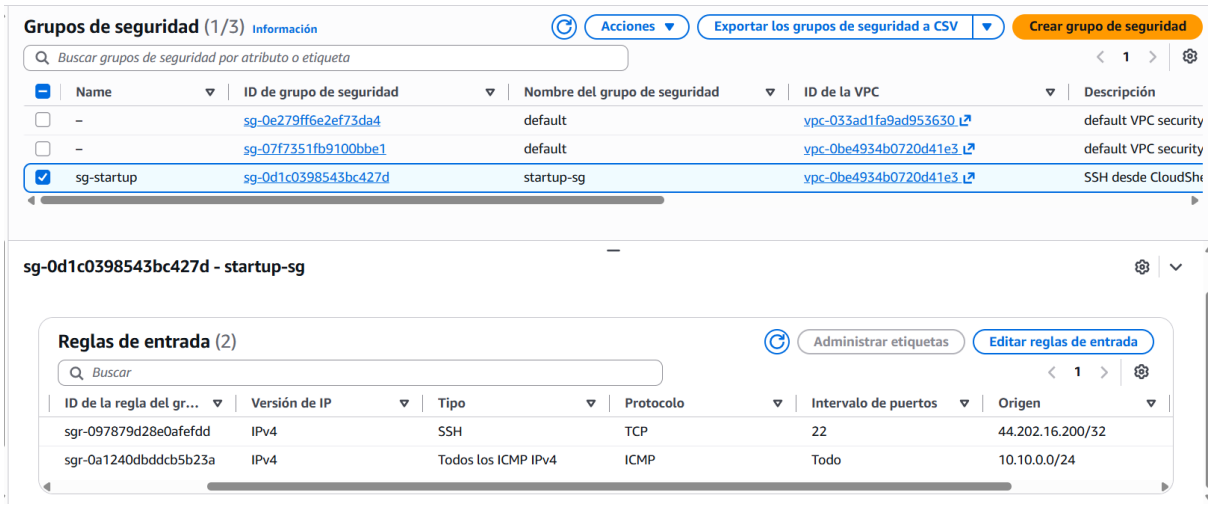
Distribución de las subredes, tal como se planearon en la parte teórica.



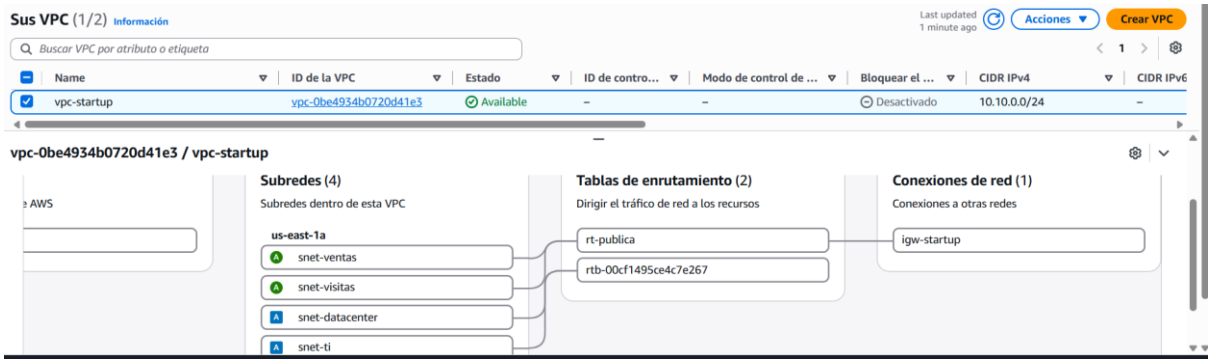
Estado de las instancias de VMs dentro de AWS. Cada una simula una parte de la lógica de negocio (distintas IPs para hacer el ping).



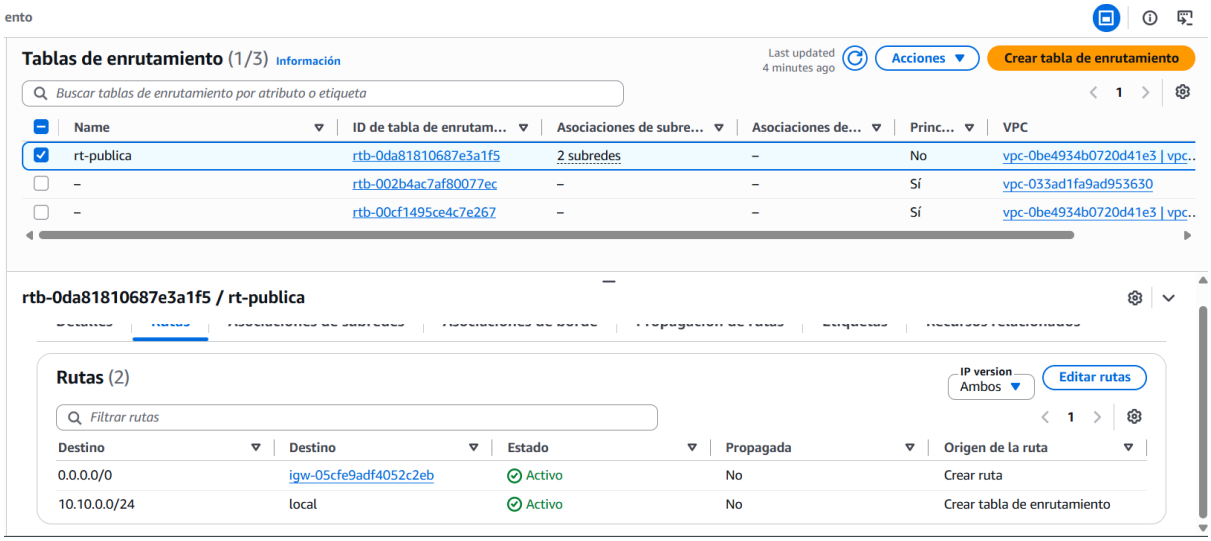
Forma del ACL de red, donde se ve las reglas que tienen las distintas IPs



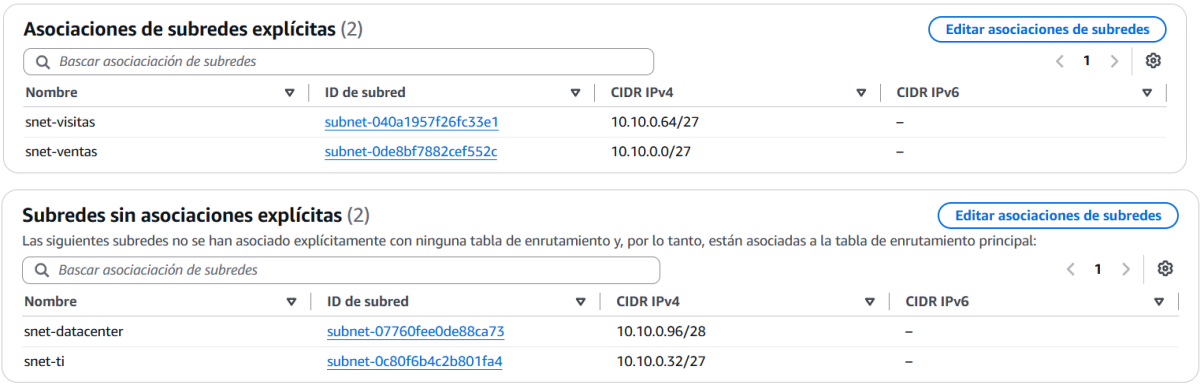
Grupos de seguridad, para ver la conexión de SSH autorizada.



Rutas que siguen las subredes para salir a internet.



Configuración de tablas de enrutamiento



Evidencia de cuáles subredes tienen acceso a salir a internet.

6. Pruebas de conectividad

```
~ $ bash pruebas-aws.sh
vm-ventas-1 = 10.10.0.15 (publica 3.239.67.207)
vm-ventas-2 = 10.10.0.9 (publica 34.201.26.98)

##### PRUEBA 1: vm-ventas-1 -> vm-ventas-2 (misma subred, DEBE funcionar)
ip-10-10-0-15.ec2.internal
10.10.0.15
PING 10.10.0.9 (10.10.0.9) 56(84) bytes of data.
64 bytes from 10.10.0.9: icmp_seq=1 ttl=127 time=0.771 ms
64 bytes from 10.10.0.9: icmp_seq=2 ttl=127 time=0.268 ms
64 bytes from 10.10.0.9: icmp_seq=3 ttl=127 time=0.212 ms
64 bytes from 10.10.0.9: icmp_seq=4 ttl=127 time=0.215 ms

--- 10.10.0.9 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3111ms
rtt min/avg/max/mdev = 0.212/0.366/0.771/0.234 ms
```

Prueba del primer ping, que va desde la primera VM de ventas hasta la segunda, donde ambas se encuentran bajo la misma subred. Como se observa, funciona correctamente

```
##### PRUEBA 2: vm-ventas-2 -> vm-ventas-1 (sentido inverso, DEBE funcionar)
ip-10-10-0-9.ec2.internal
10.10.0.9
PING 10.10.0.15 (10.10.0.15) 56(84) bytes of data.
64 bytes from 10.10.0.15: icmp_seq=1 ttl=127 time=0.203 ms
64 bytes from 10.10.0.15: icmp_seq=2 ttl=127 time=0.295 ms
64 bytes from 10.10.0.15: icmp_seq=3 ttl=127 time=0.248 ms
64 bytes from 10.10.0.15: icmp_seq=4 ttl=127 time=0.211 ms

--- 10.10.0.15 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3141ms
rtt min/avg/max/mdev = 0.203/0.239/0.295/0.036 ms
```

Segunda prueba de conectividad es el mismo anterior, pero en dirección contraria, comprobando el correcto ruteo.

```
##### PRUEBA 3 (bonus): vm-visitas-1 (10.10.0.81) -> vm-ventas-1 (DEBE FALLAR: la NACL aisla Visitas)
ip-10-10-0-81.ec2.internal
10.10.0.81
PING 10.10.0.15 (10.10.0.15) 56(84) bytes of data.

--- 10.10.0.15 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3135ms
```

Tercera prueba realizada, con intento de conexión entre visitas y ventas; pues visitas está excluida de cualquier tipo de conexión con las demás subredes, pues es la que está totalmente abierta al público.

7. Archivos de configuración y documentación

En GitHub se encuentran tanto los 3 scripts de configuración, pruebas y limpieza de las instancias, así como el resultado de haber corrido dichos scripts, con resultados en formato JSON, junto con las especificaciones de las instancias.

Link al repositorio de GitHub:
https://github.com/melisadmendizabal/redes_proyecto2